

Bases de données avancées

Déclencheurs (Triggers)



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma



Plan du chapitre

Introduction

- Qu'est ce qu'un déclencheur?
- Définitions & généralités
- Utilités d'un déclencheur
- Mécanisme général

Types de Triggers

- Contexte général
- Triggers selon l'évènement déclencheur
- Triggers selon le moment d'exécution
- Triggers suivant le niveau de granularité

Création d'un trigger

- Syntaxe générale
- Explications & remarques
- Noms de corrélations NEW et OLD
- Le trigger INSTEAD OF

Opérations sur les triggers

- Activer/Désactiver un trigger
- Informations sur un trigger
- Conclusion générale

1

Introduction



Qu'est ce qu'un déclencheur?

- On les appelle en anglais: **Triggers**
- Un trigger et une séquence d'actions, définie par le programmeur, qui se déclenche lors de l'occurrence d'un **événement particulier**
- L'exécution d'un déclencheur n'est pas explicitement opérée par une commande ou dans un programme, c'est l'évènement (mise à jour de la table par exemple) qui exécute automatiquement le code programmé dans le déclencheur.

Les triggers sont de simples procédures stockées qui s'exécutent implicitement lorsqu'une instruction INSERT, DELETE ou UPDATE porte sur la table.



Généralités

- Les déclencheurs existent depuis la version 6 d'Oracle.
- Ils sont compilables depuis la version 7.3 (auparavant, ils étaient évalués lors de l'exécution).
- Version 8 : apparition des déclencheurs **INSTEAD OF** permet la mise à jour de vues multitables
- Les déclencheurs ont un **nom**, une partie **déclaration**, un **corps** et une partie **exception**.
- Les déclencheurs n'acceptent pas d'arguments ni peuvent être tirés explicitement.
- Les déclencheurs s'exécutent **automatiquement** par Oracle d'une manière **transparente** à l'utilisateur.

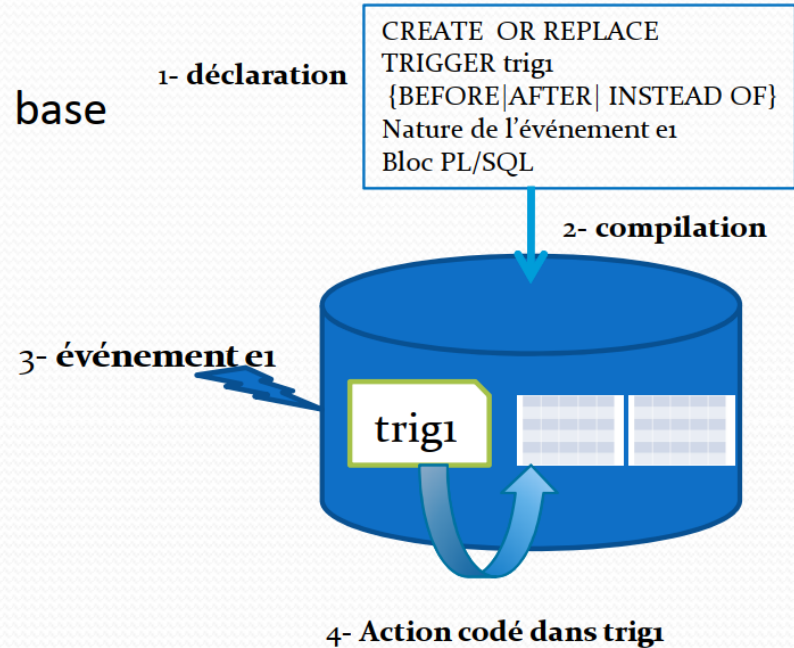


Macanisme général

Les étapes :

- coder le déclencheur
- le compiler == > il sera stocké ainsi en base

➔ si le déclencheur est actif
chaque événement qui caractérise
le déclencheur aura pour
conséquence son exécution



2

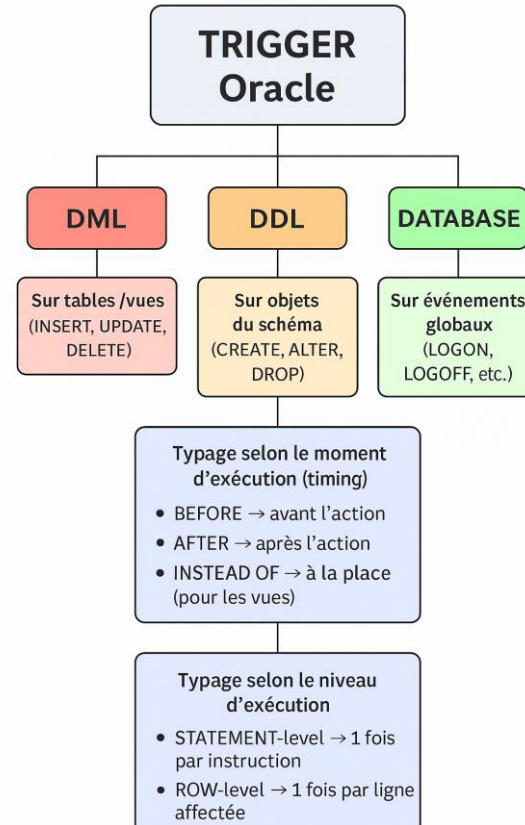
Types de triggers



Contexte général



En PL/SQL, les triggers peuvent être classés selon plusieurs dimensions, notamment le **type d'événement** déclencheur, le **moment d'exécution** et la **granularité**.





Selon l'évènement déclencheur

Les triggers sont déclenchés par différents types d'événements :

- **Triggers DML** (Data Manipulation Language) : Déclenchés par les instructions INSERT, UPDATE, DELETE sur une table ou une vue. On peut spécifier une liste de colonnes pour les triggers UPDATE (par exemple, UPDATE OF salaire).
- **Triggers DDL** (Data Definition Language) : Déclenchés par les instructions CREATE, ALTER, DROP, RENAME, etc., au niveau d'un schéma ou de la base de données.
- **Triggers système** (Database Events) : Déclenchés par des événements de base de données ou d'utilisateur comme SERVERERROR, LOGON, LOGOFF, STARTUP, SHUTDOWN



Selon timing

Cela détermine si le trigger s'exécute avant ou après l'événement déclencheur :

- **BEFORE triggers** : S'exécutent avant l'instruction SQL déclenchante. Ils sont souvent utilisés pour valider des données, générer des valeurs de colonnes dérivées ou empêcher l'exécution de l'instruction si une condition n'est pas remplie.
- **AFTER triggers** : S'exécutent après l'exécution de l'instruction SQL déclenchante et après la vérification des contraintes d'intégrité. Ils sont utiles pour l'audit, la journalisation des événements ou la mise à jour de tables connexes.
- **INSTEAD OF triggers** : Utilisés pour les vues qui ne sont pas directement modifiables. Le trigger s'exécute à la place de l'instruction DML (INSERT, UPDATE ou DELETE) et contient la logique PL/SQL nécessaire pour modifier les tables sous-jacentes de la vue.



Selon la granularité

Cela définit le nombre de fois que le corps du trigger est exécuté :

- Triggers de niveau **ligne** (FOR EACH ROW) : Le trigger se déclenche une fois pour chaque ligne affectée par l'instruction SQL. Ils ont accès aux valeurs anciennes (:OLD) et nouvelles (:NEW) de la ligne en cours de traitement.
- Triggers de niveau **instruction** (par défaut, sans FOR EACH ROW) : Le trigger se déclenche une seule fois pour l'ensemble de l'instruction SQL, quel que soit le nombre de lignes affectées (même si aucune ligne n'est affectée).

3

Création d'un trigger



Syntaxe de création

- On utilise la commande **CREATE TRIGGER** pour créer un déclencheur d'une base de données.
- On utilise aussi:
 - Nom du trigger,
 - La table associée,
 - Quand un déclencheur est tiré: before ou after
 - La commande qui invoque le déclencheur: update, delete ou insert (ou plusieurs)
 - Condition de filtrage
 - Le bloc PL/SQL qui sera exécuté quand le déclencheur s'est tiré



Syntaxe de création

- Création du trigger

CREATE [OR REPLACE] TRIGGER nom_trigger

{BEFORE|AFTER | INSTEAD OF}

nom-trigger :doit être unique dans un même schéma

Pour les vue

table ou vue associé au déclencheur

**)

{INSERT|DELETE|UPDATE [OF col1 ..]} ON <[schéma.]nomTable|nomVue>

[FOR EACH ROW]

[WHEN (<condition>)]

DECLARE <<<<déclarations>>>>

BEGIN

..... <<<< bloc d'instructions PL/SQL>>>>

END;

nature de l'événement

Cœur du trigger

précise le moment de l'exécution du trigger

**) Pour UPDATE, on peut spécifier une liste de colonnes. Dans ce cas, le trigger ne se déclenchera que si l'instruction UPDATE porte sur l'une au moins des colonnes précisée dans la liste



Explications

- **CREATE OR REPLACE**: Recréer le déclencheur s'il existe déjà.
- **BEFORE|AFTER**: Avant ou après l'événement déclenchant le trigger, qui peut être une insertion, suppression ou mise à jour (INSERT|DELETE|UPDATE) sur une table.
- L'option **FOR EACH** fait exécuter le trigger à chaque modification d'une ligne de la table spécifiée (on dit que le trigger est de "**niveau-ligne**").
- En l'absence de cette option, le trigger est exécuté une seule fois ("**niveau table**").
- **ROW [WHEN (condition)]** est utilisé pour ne déclencher le trigger que si la condition est vérifiée.
- L'option **OF** permet de spécifier les colonnes concernées pour déclencher le trigger.
- **<corps du trigger>** représente un bloc de code à exécuter au déclenchement d'un trigger



Remarques sur la création de Triggers

- la taille d'un déclencheur ne peut excéder 32 ko == > limiter la taille (partie instructions) d'un déclencheur à soixante lignes de code PL/SQL

=> contournement : appeler des sous- programmes dans le code du déclencheur (fonctions et procédures).

- Un déclencheur ne peut valider aucune transaction == > les instructions suivantes sont interdites : COMMIT, ROLLBACK, SAVEPOINT
- Attention à ne pas créer de déclencheurs récursifs (exemple d'un déclencheur qui exécute une instruction lançant elle-même le déclencheur ou deux déclencheurs s'appelant en cascade jusqu'à l'occupation de toute la mémoire réservée)



Exemple

Testons ce trigger qui affiche un message si une nouvelle insertion dans la table EMPLOYE est détectée

```
CREATE OR REPLACE TRIGGER trg_before_insert_emp
BEFORE INSERT ON employe
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Insertion sur EMPLOYE détectée.');
```



Les noms de **corrélation**

- Lors de la création de triggers lignes (Row-level), il est possible d'avoir accès à la valeur ancienne et la valeur nouvelle grâce aux mots clés **OLD** et **NEW**.
- Il n'est pas possible d'avoir accès à ces valeurs dans les triggers de table (Statement-level).
- Si l'instruction de déclenchement du trigger est INSERT, seule la nouvelle valeur a un sens.
- Si l'instruction de déclenchement du trigger est DELETE, seule l'ancienne valeur a un sens.
- La nouvelle valeur est appelée **:new.colonne** et l'ancienne valeur est appelée **:old.colonne**

Exemple : IF :new.salaire < :old.salaire then

	les nouvelles valeurs sont dans	les anciens valeurs sont dans
déclencheur sur INSERT,	:new.<nom attribut>	Pas d'accès à l'élément OLD (qui n'existe pas)
déclencheur sur UPDATE	:new.<nom d'attribut>	:old.<nom d'attribut>.
déclencheur sur DELETE	Pas d'accès à l'élément NEW (qui n'existe plus)	:old.<nom d'attribut



Exemple de NEW

-1



Contrainte : un pilote ne doit pas être qualifié sur plus de trois types d'avions

Pilote				
Brevet	nom	nbhvol	comp	nbqualif
PL-1	J.M toto	450	AF	3
PL-2	T. Guibert	3400	AF	1
	Michel			
PL-3	Tuf	900	SING	1

Qualifications

brevet	typeA	expire
PL-1	A340	22/06/2005
PL-1	A340	05/02/2005
PL-1	A320	16/01/2004
PL-2	A320	18/01/2004
PL-3	A330	22/01/2006



Question : comment gérer des insertions dans la table Qualifications pour répondre à la contrainte ci-dessus?



Exemple de NEW

-2



L'exécution de cette instruction: **insert into Qualifications values('PL-2','A380','20-06-2006');** doit déclencher le trigger ci-après

```
CREATE OR REPLACE TRIGGER T_INSQualif
  BEFORE INSERT ON Qualifications
  FOR EACH ROW
  DECLARE
    v_compteur Pilote.nbhvol%type;
    v_nom Pilote.nom%type;
  BEGIN
    SELECT nbqualif, nom into v_compteur, v_nom from Pilote where brevet=:New.brevet;
    if v_compteur < 3 THEN
      Update Pilote set  nbqualif= nbqualif + 1
      where brevet=:New.brevet;
    else
      RAISE_APPLICATION_ERROR(-20100, 'le pilote' || v_nom || ' a déjà 3 qualifs');
    END IF;
  END;
```

Événement : before insert



Exemple de OLD

```
CREATE TRIGGER T_delQualif
AFTERE DELET ON Qualifications
FOR EACH ROW
BEGIN
    UPDATE Pilote set nbqualif =nbqualif-1
    where   brevet=:OLD.brevet;
END;
```



Exemple de WHEN



Possibilité de restreindre l'exécution du trigger

```
CREATE OR REPLACE TRIGGER T_INSQualif
BEFORE INSERT ON Qualifications
FOR EACH ROW
WHEN (NEW.typeA='A320' OR NEW.typeA='A340')
DECLARE
    v_compteur Pilote.nbhvol%type;
    v_nom Pilote.nom%type;
BEGIN
    SELECT nbqualif, nom into v_compteur, v_nom from Pilote where
    brevet=:New.brevet;
    if v_compteur <3 THEN
        Update Pilote set nqualif= nqualif + 1 where brevet=:New.brevet;
    else
        RAISE_APPLICATION_ERROR(-20100, 'le pilote' || v_nom ' a déjà 3 qualifs');
    END IF;
END;
```



Trigger **INSTEAD OF**

- Ces triggers sont définis pour des vues.
- Ils sont utilisés pour modifier des vues qui ne peuvent pas être modifiées par des commandes DML.
- Au contraire des triggers normaux qui sont déclenchés lors de l'exécution d'une commande DML, ceux-ci se déclenchent au lieu d'exécuter ces commandes.
- Ils ne font pas intervenir les options BEFORE et AFTER.
- Pas possible de spécifier une liste de colonnes dans un déclencheur INSTEAD OF UPDATE



Exemple **INSTEAD OF** -1



```
CREATE VIEW VueCompPil  
AS SELECT c.comp, c.nomcomp, p.brevet,  
p.nom, p.nbHvol  
FROM Pilot p, Compagnie c  
WHERE p.comp = c.comp
```

Brevet	nom	nbhvol	comp
PL-1	J.M toto	450	AF
PL-2	T. Guibert	3400	AF
PL-3	MichelTuf	900	SING

comp	nrue	rue	ville	nomComp
AF	124	rue 1	Paris	Air France
SING	7	rue2	Singapour	Singapore AL

```
== > insert INTO Vue VueCompPil  
      Values ('AERIS', 'Aéris Toulouse','PL4','Pascal Larrazet', 5600);  
== > le déclencheur qui gère les insertion dans la vue est chargé d'insérer , à  
      chaque nouvel ajout, un enregistrement dans chacun des deux tables
```




Exemple **INSTEAD OF** -2

```
CREATE OR REPLACE TRIGGER T_VueCompPil
INSTEAD OF INSERT ON VueCompPil
FOR EACH ROW
DECLARE
    V_comp NUMBER;
    V_pil NUMBER;
BEGIN
    select count(*) into v_pil from Pilote where brevet=:new.brevet
    select count(*) into v_comp from compagnie where comp=:new.comp
    if(v_pil >0) then
        RAISE_APPLICATION_ERROR(-20103, 'le pilote existe déjà');
    else if v_pil=0 THEN
        insert into Pilote values(:new.brevet, :new.nom, ...);
    End if;

    IF v_comp=0 THEN insert into Compagnie values(:new.comp, null,null, ...,
    new.nomComp);
    ELSE RAISE_APPLICATION_ERROR(-20103, 'la compagnie existe déjà');

END;
```

4

Opérations sur les triggers



Activer / Désactiver

Désactivation

- Par défaut, un trigger est activé dès sa création.
- Désactiver un trigger peut améliorer la performance lorsqu'une grande quantité de données d'une table doit être modifiée.
- Pour désactiver le trigger : **ALTER TRIGGER nomTrigger DISABLE;**
- Pour désactiver tous les triggers sur une table : **ALTER TABLE nomTable DISABLE ALL TRIGGERS;**

Activation

- Pour activer un trigger : **ALTER TRIGGER nomTrigger ENABLE;**
- Pour activer tous les triggers sur une table : **ALTER TABLE nomTable ENABLE ALL TRIGGERS;**



Informations sur les triggers



Recherche d'information sur les triggers

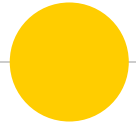


Les définitions des triggers sont stockées dans : **USER_TRIGGERS**, **ALL_TRIGGERS** et **DBA_TRIGGERS**



Conclusion

- Les déclencheurs d'une base de données sont utilisés pour implémenter des règles compliquées qu'on ne peut pas exprimer avec les contraintes d'intégrité.
- Un trigger peut être déclenché avant ou après une opération DML. Il peut être déclenché pour chaque ligne affectée par une opération DML ou pour toutes les instructions à la fois.
- Le trigger `INSTEAD-OF` est appelé au lieu d'exécuter une commande DML donnée. Il est défini pour des vues.
- Un trigger peut être activé ou désactivé pour augmenter la performance quand beaucoup de manipulations des données sont à faire sur la base de données



Fin du chapitre 6